

Aula 11

Ministrado 20/05/2025

Álgebra Relacional

Conjuntos Compatíveis

Conjunto cujo os elementos tem o mesmo número e são do mesmo tipo.

União - $\cup$

Todas as tuplas em A , B ou em ambas, sem repetição.

$A \cup B$, somente se A e B forem compatíveis
cliente(RG , nome.cli , datanasc , saldo , sexo)
agencia(num.ag , nome.ag , cidade.ag , ativo)
deposito(num.dep , RG , saldo , conta.ag)
emprestimo(num.empr , RG , conta.ag , sexo)

Assim as únicas que poderia ser unido é o conjunto deposito e o conjunto emprestimo.

Intersecção - $\cap$

Todas as tuplas que estão em A e B ao mesmo tempo.

Subtração - $-$

Todas as tuplas que estão em uma relação porém não estão em outra.

Junção Natural - $\bowtie$

Combinação de tuplas relacionadas de duas relações em uma única tupla.

Pode ser definida como um produto cartesiano seguido de uma seleção.

SQL - Structured Query Language

Padrão adotado para linguagens de consultas de banco de dados em Sistemas Gerenciadores de Bancos de Dados Relacionais.

Com o uso de cláusulas SQL é possível realizar tarefas como definir a estrutura das tabelas; manipular os dados armazenados – inserir, eliminar e consulta-los; entre outras.

Os SGBDRs implementam variações do padrão SQL, incluindo particularidades e funcionalidades adicionais.

Os componentes básicos da SQL geralmente são comuns a todos os SGBDRs

Entre os mais usados são: MySQL, Microsoft SQL Server, Oracle, PostgreSQL, DB2, Firebird.

Consulta SQL

SELECT, FROM, WHERE

A estrutura básica para realizar consultas consiste basicamente em três cláusulas, **select**, **from** e **where**.

A clausula **FROM** pode receber uma lista de relações.

Deve ser informada todas as relações que possuem informações que deseja-se obter

Utilizar a cláusula **WHERE** para especificar a condição para combinar as informações.

```
SELECT < lista de atributos >  
FROM < lista de relacoes > [WHERE < predicado >]
```

Um exemplo pode ser visto abaixo:

```
SELECT nome  
FROM cliente
```

Para evitar a exibição de resultados duplicados utiliza-se o comando **distinct**.

```
SELECT DISTINCT nome_cidade,  
FROM cliente
```

Para retornar apenas uma seleção dos dados é possível utilizar a cláusula **WHERE** para filtrar os resultados e exibir apenas aqueles de interesse.

```
SELECT nome_cli  
FROM cliente  
WHERE sexo = 'M' and ano_nasc = 1984
```

Na consulta acima podemos relacionar os elementos ao seguinte conjunto de operações de Álgebra Relacional.

$$\Pi_{nome.cli}(\sigma_{sexo=M \cap ano.nasc=1984}(cliente))$$

IN

É possível aplicar operações de conjunto em uma consulta SQL, por meio do operador IN.

```
SELECT *
```

```
FROM cliente
```

```
WHERE RG IN
```

```
      (SELECT RG
```

```
        FROM deposito
```

```
        WHERE valor > 500)
```

UNION

O comando UNION é compatível somente se apresentar o mesmo tipo de domínio, apesar de ser composto por atributos diferentes.

```
(SELECT num_dep,
```

```
      saldo
```

```
FROM deposito
```

```
WHERE cod_ag=101)
```

```
UNION
```

```
(SELECT num_empr,
```

```
      valor
```

```
FROM emprestimo
```

```
WHERE cod_ag=101)
```

INTERSECT

O comando INTERSECT possui o mesmo funcionamento do UNION.

```
(SELECT RG
  FROM deposito
 WHERE cod_ag=101 and valor>200,00) INTERSECT
(SELECT RG
  FROM emprestimo
 WHERE cod_ag=101)
```

MINUS

O comando MINUS possui o mesmo funcionamento do UNION.

```
(SELECT num_dep, saldo
  FROM deposito
 WHERE cod_ag = 101 AND saldo >= 200,00) MINUS
      (SELECT num_empr, valor
        FROM emprestimo
       WHERE cod_ag = 101)
```

AS

Conceito de variáveis tuplas, atribuindo a uma mesma relação vários identificadores.

```

SELECT X.RG,
       X.nome_cli,
       X.data_nasc
FROM cliente AS X,
     cliente AS Y
WHERE Y.nome = 'Fernando'
     AND X.cidade_cli = Y.cidade_cli

```

SOME

Utilizado para comparar valores com conjuntos, sendo eles <, =, <=, >=, <>, =

```

SELECT DISTINCT RG
FROM deposito
WHERE saldo > SOME
      (SELECT saldo
       FROM deposito
       WHERE RG = 3832012)

```

ALL

Usado para comparar valor com conjuntos

```
(SELECT x.RG, x.nome_cli, x.cidade_cli
FROM cliente AS x
WHERE data_nasc <= ALL
      (SELECT y.data_nasc
      FROM cliente As y
      WHERE x.cidade_cli=y.cidade_cli))
```